



Portfolio

이유진 Frontend Engineer

사용자가 구매 흐름에서 느끼는 작은 마찰을 발견하고, 성능, 인증, 데이터 수집 구조를 수치 기반으로 개선하는 프론트엔드 개발자 이유진입니다.

디자인과 구현 사이의 디테일을 중요하게 보며, 몇 ms의 입력 지연, 로그인 직후의 인증 유실, 클릭 이벤트 누락처럼 전환을 방해하는 문제를 끝까지 추적해 개선합니다.

 장바구니 가격 반영 시간 1.5초 → 0.6초 개선

 iOS WKWebView 로그인 직후 401 오류 88% 감소

 웹/모바일 52개 페이지 이벤트 추적 구조 표준화

 AI 기반 자동화로 배포 후 릴리즈 노트 정리 시간 10분 → 1분 단축

Featured

1. 구매 전환 플로우 성능 및 안정성 개선

떠리몰 모바일 웹과 웹 서비스에서 상품 탐색, 로그인, 장바구니 등 구매 전환에 직접 연결되는 사용자 흐름을 개선했습니다.

외주 개발 이관 이후 복잡하게 분산되어 있던 이커머스 비즈니스 로직을 안정화하고, 사용자가 실제로 체감하는 지연과 실패 지점을 수치 기반으로 줄이는 데 집중했습니다.



주요 성과

- 장바구니 가격 반영 시간 1.5초 → 0.6초 개선
- iOS WKWebView 로그인 직후 5분 내 401 오류 88% 감소

1-1. 장바구니 수량 변경 UX 개선

Problem

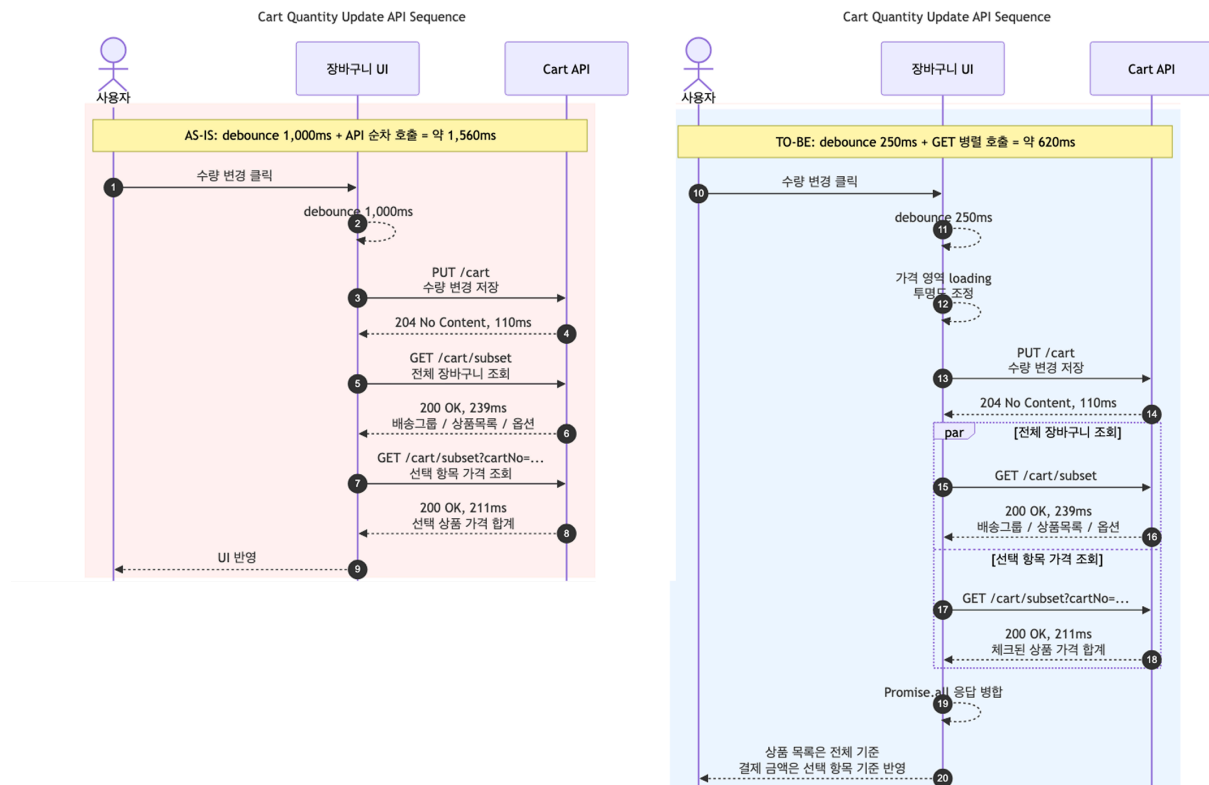
장바구니에서 수량을 변경하면 가격이 반영되기까지 약 **1.5초** 가 걸렸습니다. 수량을 여러 번 조정할 때 변경 결과가 늦게 보여 사용자가 기다려야 했고, 구매 흐름이 끊기는 문제가 있었습니다.

Cause

수량 변경 이후 장바구니 전체 데이터와 선택 항목 기준 가격 집계 데이터를 갱신하는 과정이 반복적으로 발생했습니다. 기존에는 회원/비회원, 모바일/웹 로직이 화면별로 흩어져 있어 같은 문제를 수정할 때 여러 구현을 함께 확인해야 했습니다.

Solution

- 전체 장바구니 데이터와 선택 항목 기준 집계 가격 조회를 **Promise.all** 로 병렬 처리
- 수량 변경 핸들러를 공통 **useCartQty** 훅으로 분리하고 debounce를 적용해 연속 클릭 시 API 호출량 제어
- 모바일/PC, 회원/비회원에게 나뉘어 있던 장바구니 수량 변경 로직을 공통 함수로 분리



Result

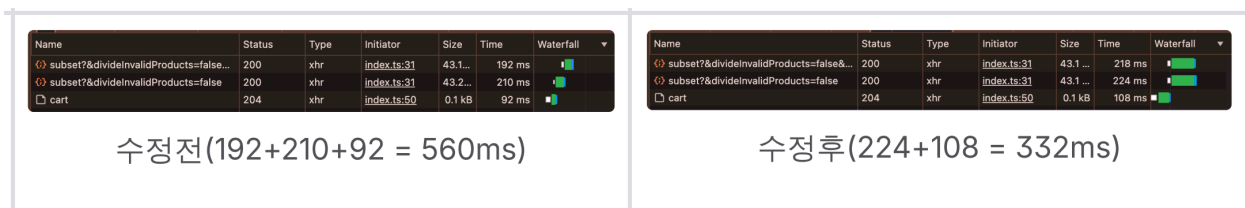
- 가격 반영 시간 1.5초 → 0.6초 개선
- 응답 시간 약 60% 단축
- 중복 로직을 줄여 및 장바구니 유지보수성 개선

Learned

몇 ms 단위의 조정이 만드는 체감 UX 차이

장바구니 수량 변경 debounce를 1000ms → 250ms 로 줄이며, 구매 전환 구간에서는 API 요청 수뿐 아니라

사용자가 가격 변화를 즉시 확인할 수 있는지도 함께 봐야 한다는 기준을 갖게 되었습니다.



병렬처리 전/후 네트워크 속도 비교

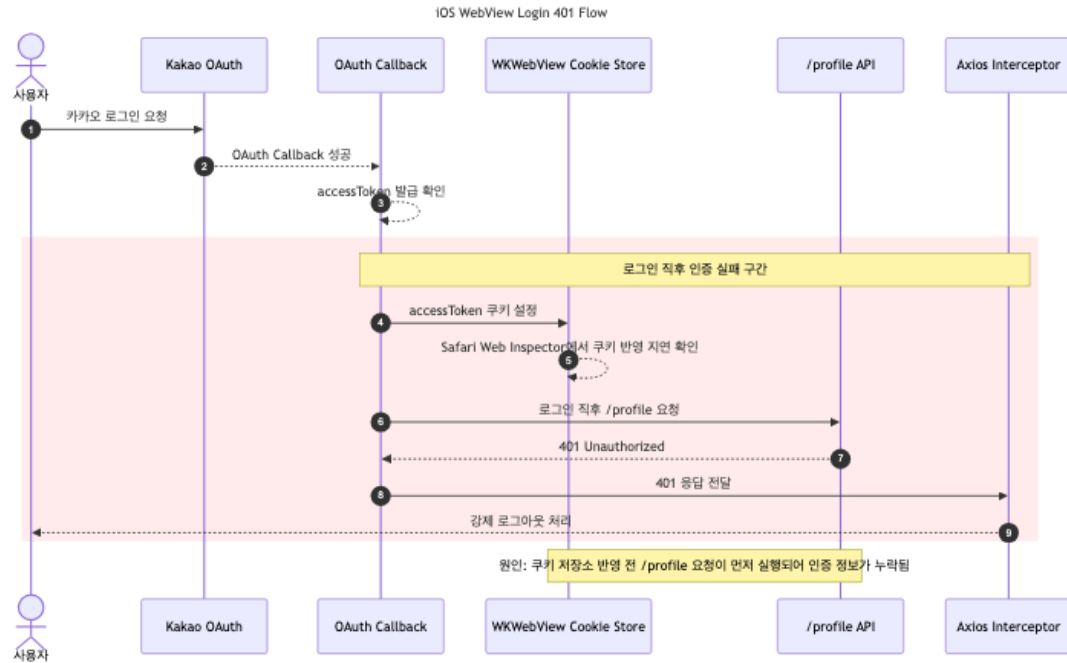
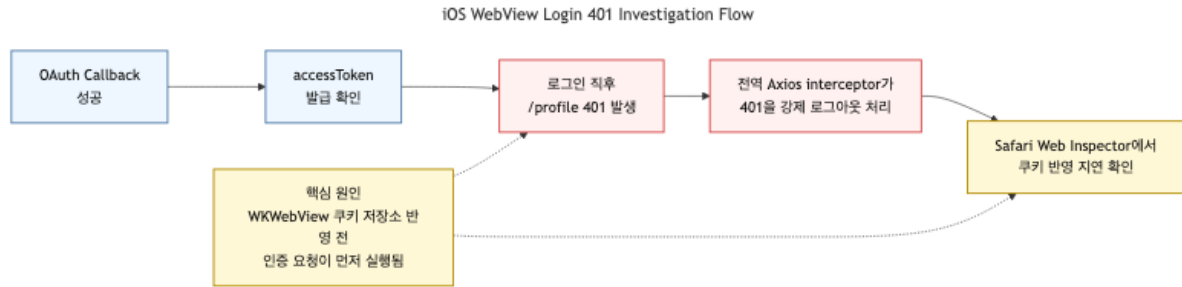
1-2. iOS WebView 로그인 401 오류 개선

Problem

- iOS WebView 환경에서 카카오 로그인 직후 401 오류가 반복적으로 발생했습니다. 사용자는 로그인에 성공한 것처럼 보였지만, 곧바로 인증이 풀리거나 강제 로그아웃되는 경험을 했습니다.

Investigation

- OAuth 스펙 기준으로 서비스 ↔ 샵바이 SaaS ↔ 카카오 로그인 프로바이더 인증 흐름 정리
- OAuth Callback, accessToken 발급, /profile 인증 요청 구간의 로그를 단계별로 분리
- Safari Web Inspector로 iOS WKWebView의 쿠키 및 인증 헤더 상태 확인
- 로그인 성공 직후 accessToken 발급은 성공하지만, 일부 iOS WebView에서 /profile 요청 시 인증 쿠키가 누락되는 패턴 확인
- 401 로그에 요청 URL, memberNo, accessToken 존재 여부를 함께 기록해 세션 만료와 WebView 쿠키 반영 지연 가능성을 비교 분석할 수 있는 근거를 확보



Solution

- OAuth Callback 응답 직후 클라이언트에서도 `accessToken` 쿠키를 명시적으로 설정해 서버 응답 쿠키와 클라이언트 쿠키 상태를 동기화하였습니다.
- 전역 axios interceptor가 모든 401을 즉시 로그아웃으로 처리하던 흐름을 점검
- 로그인 Callback → 토큰 발급 → /profile 요청 → 401 로그아웃 지점을 단계별로 추적할 수 있도록 로그 보강
- iOS WKWebView에서 쿠키 반영 지연이 의심되는 로그와 재현 흐름을 애플에 공유하고, WebView 설정 점검 및 다음 앱 배포 반영을 요청

Result

- 로그인 직후 5분 내 401 인증 오류 `88%` 감소 (`170건` → `20건`)
- `iOS WKWebView` 환경의 로그인 유지 안정성 개선

Learned

WebView 인증은 쿠키 반영 타이밍까지 검증해야 한다

OAuth Callback 성공 직후 인증 요청에서 `accessToken` 이 누락되는 문제를 추적하며, `iOS WKWebView` 에서
는 쿠키 저장소 반영 시점에 따라 브라우저와 다른 인증 실패가 발생할 수 있음을 확인했습니다.

서버 `Set-Cookie` 흐름을 유지하는 것이 원칙이지만, 해당 이슈는 로그인 직후 타이밍 문제였기 때문에
Callback 구간에 한정해 클라이언트 쿠키 설정을 보완 적용했습니다.

3.1 프론트엔드 로그 시퀀스

```
1 [SNS_LOGIN][AuthCallback] /api/callback 호출 시작 - params: kakao, code: exists,  
  openAccessToken: null  
2 [SNS_LOGIN][AuthCallback] /api/callback 응답 - status: 200, ok: true  
3 [SNS_LOGIN][AuthCallback] 응답 데이터:  
  {"hasProfile":true,"hasAccessToken":true,"hasAccount":false}  
4 [SNS_LOGIN][AuthCallback] flutter callHandler("login") 호출 완료, result: Promise {status:  
  "pending"}  
5 [SNS_LOGIN][AuthCallback] 게스트 장바구니 병합 시작  
6 [SNS_LOGIN][AuthCallback] Redux login dispatch 시작/완료  
7 [SNS_LOGIN][AuthCallback] 리다이렉트 판단 - guestPayment: false, guestBuy: false, state: /my-  
  page  
8 [SNS_LOGIN][AuthCallback] 일반 리다이렉트 → /my-page  
9 — 여기까지 "성공" 흐름 —  
10 Failed to load resource: the server responded with a status of 401 () https://shop-api.e-  
  ncp.com/cart/count  
11 401 Unauthorized(mobile axios): {path: "/cart/count", status: 401, code: "NCPE0003",  
  message: "인증 정보가 필요합니다."}  
12   access_token: "5FL6g1yvwC3ZdespQC5Z0KuoU-E" ← 401 핸들러 시점 쿠키 값 (요청 헤더 아님)  
13   memberNo: 121593025
```

3.2 Safari Web Inspector — Storage 탭

- `access_token`, `user` — 없음 (401 핸들러의 `logout()` 이 지움)
- `last_login_provider` — 있음 (logout 리듀서가 삭제하지 않는 쿠키라서 흔적으로 남음)

3.3 수동 curl 검증

```
1 curl -H 'access_token: 5FL6g1yvwC3ZdespQC5Z0KuoU-E' https://shop-api.e-ncp.com/cart/count  
2 # → {"count": 7}
```

토큰 자체는 유효. 요청 시점에 interceptor가 토큰을 읽지 못한 것이 문제.

로그 분석 문서

2. 전시/추천 영역 이벤트 트래킹 표준화

마케팅 신규 페이지와 추천 섹션의 성과를 일관되게 측정할 수 있도록 `GTM → MMP 솔루션` 기반 이벤트 추적
구조를 표준화했습니다.

전시 영역 트래킹 스펙을 문서화하고, 산발적으로 호출되던 마케팅 SDK 연동 코드를 공통 인터페이스 기반으로 분리했습니다.



주요 성과

- 웹/모바일 52개 페이지 추적 구조 표준화
- `page → section → item` 기반 이벤트 위계 정의
- 신규 페이지와 섹션의 이벤트 구현 방식 통일
- MMP 솔루션 트래킹 호출을 `packages/common` 공통 provider로 분리

2-1. 전시 영역 트래킹 스펙 및 이벤트 위계 표준화

Problem

전시 영역 트래킹 구조가 체계화되어 있지 않아 페이지와 섹션마다 이벤트 수집 방식과 데이터 형식이 달랐습니다. 때문에 기획/마케팅팀이 성과를 일관되게 비교하기 어려웠습니다.

Solution

- `page → section → item` 기반 이벤트 위계 정의 및 문서화
- `data-component`, `data-id`, `data-idx`, `data-label` 등 공통 데이터 스키마 정의
- 웹/모바일에서 같은 규칙으로 태그를 적용할 수 있는 `DataTagWrapper` 컴포넌트 구축
- 신규 페이지 작업자가 같은 기준으로 구현할 수 있도록 스펙 문서와 예시 코드 공유

Result

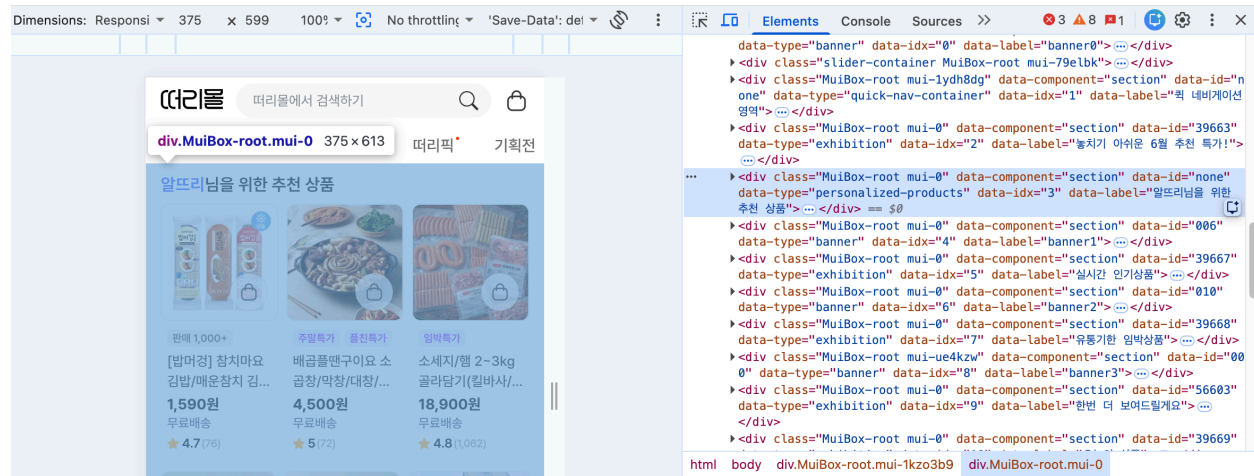
- 페이지/섹션별로 달랐던 이벤트 데이터 형식을 공통 스키마로 통일
- 웹/모바일 52개 페이지의 전시 트래킹 구조 표준화
- 기획/마케팅팀이 `page > section > item` 단위로 성과를 비교할 수 있는 기준 마련

Learned

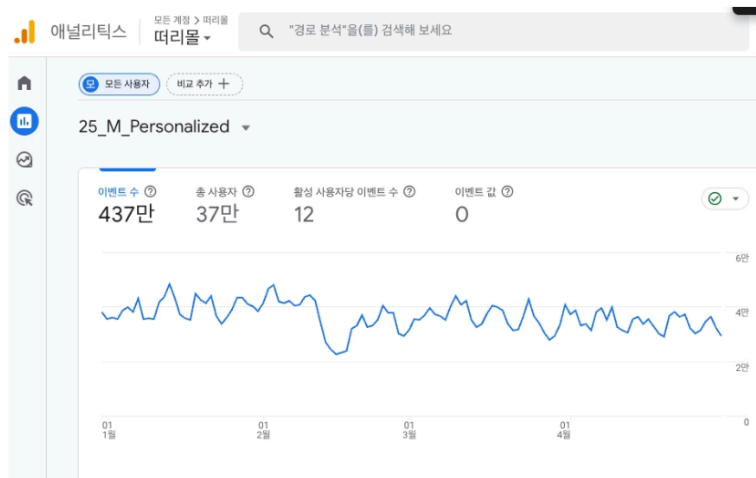
트래킹은 이벤트 추가가 아니라 분석 가능한 구조를 설계하는 일이었다

개별 이벤트 구현은 페이지가 늘어날수록 명세와 실제 코드가 어긋나기 쉬워, `page → section → item` 위계의 공통 트래킹 구조로 정리했습니다.

스펙 문서와 예시 코드를 함께 공유해 동료의 적용 비용을 줄이고, 기획/마케팅팀이 공통 **data** 속성 기준으로 페이지, 섹션 성과를 비교할 수 있게 했습니다.



HTML data 속성



GTM 트리거(25_M_Personalized)를 통해 GA4에서 분석

2-2. 마케팅 SDK 의존성 분리 및 트래킹 확장 구조 설계

Problem

마케팅 이벤트 호출이 페이지 코드 곳곳에 직접 섞여 있어, SDK별 이벤트명과 payload 형식 변경이 화면 코드 수정으로 이어졌습니다. MMP 솔루션 외에도 카카오, 네이버, 링크프라이스 등 외부 채널이 늘어날 가능성이 있어 화면 코드와 SDK 의존성을 분리할 필요가 있었습니다.

Solution

- 화면 코드에서는 `trackEvent(event, payload)` 만 호출하도록 공통 진입점 정의

- 외부 SDK별 이벤트명 매핑, payload 변환, 전송 로직을 `TrackingProvider` 구현체로 분리
- 현재 운영 중인 MMP 솔루션을 provider로 분리하고, 다른 채널은 provider 추가 방식으로 확장 가능하게 구성
- 공통 트래킹 로직을 `모노레포 공통 패키지(@repo/common)` 로 통합해 웹/모바일에서 동일하게 사용

Result

- 화면 코드에서는 `trackEvent` 만 호출하고, SDK별 이벤트 변환은 provider 내부에서 처리하도록 분리
- 향후 카카오, 네이버, 링크프라이스 같은 추가 마케팅 채널을 provider 단위로 확장할 수 있는 기반 마련

```
packages/common/src/lib/tracking/
  event.ts           공통 이벤트 타입 정의
  dispatcher.ts      공통 이벤트 dispatcher
  index.ts           trackEvent export

providers/
  base.ts           TrackingProvider 인터페이스
  index.ts          활성 Provider 등록
  Airbridge.ts      Airbridge Adapter 구현
    - 이벤트명 매핑
    - payload 변환
    - Airbridge.events.send() 호출
```

3. 개발 운영 자동화 및 AI 워크플로우 구축

프론트엔드 개발과 운영 과정에서 반복되는 계획 수립, PR 리뷰, 배포 검수, 릴리즈 정리, 문서화를 Claude Code Skill 기반 워크플로우로 자동화했습니다.

AI 도구를 단발성 코드 생성에만 사용하지 않고, 반복 업무를 재사용 가능한 Skill 단위로 나누어 팀에서 사용할 수 있는 운영 도구로 정리했습니다.

주요 성과

- ### 3-1. 배포 후 Jira/릴리즈 노트 정리 자동화

릴리즈 노트 정리, Jira fixVersion 지정, 이슈 상태 변경, linked work item 보고가 반복 수작업으로 진행되었습니다. 릴리즈마다 정리 기준이 달라질 수 있었고, 배포 후 문서화와 보고에 시간이 소요되었습니다.

- 운영 배포 후 정리 절차를 Claude Code Skill로 패키징해 팀원이 같은 명령으로 실행할 수 있게 구성
- 릴리즈 노트 생성, Jira 이슈 업데이트, 배포 완료 transition, linked work item 보고 자동화
- 팀원이 반복 사용할 수 있도록 명령 흐름과 사용 기준 문서화

- 릴리즈 정리 시간 10분 → 1분 단축
- 팀원 3명 사용 확산
- 배포 정리 기준 표준화

[배포] 4월 5차 업데이트

By [여유진](#) 1 min 17 Add a reaction

■ 업데이트 일시 및 범위

- 업데이트 일시: 4월 17일, 4월 20일, 4월 22일, 4월 23일, 4월 28일, 4월 29일
 - 업데이트 범위: Thirtymall PC, Thirtymall MOBILE
- 업데이트 내역

■ 업데이트 내역

[🔒 로그인 / 인증](#)

- **Mobile iOS** 강연로그인 강연객비율 수월
 - iOS에서 카카오톡이버전 2.0 강연로그인 시 1회 선택한 회원이 계속되어 첫 시도에도 정상 진행될 수 있습니다
- **Mobile iOS** 강연로그인 카카오톡 429 오류 원인 파악
 - 카카오톡 로그인도 시도 중 발생하는 오류(429)에 방화벽 경유, 사용자제가 안내 메시지까지 표시되도록 개조되었습니다.
 - 기존에는 별다른 방문 없이 로그인되어 돌아와 사용자제가 방문 알림을 받지 아였습니다.
- **Mobile** 갤럭시 노트 2021 화면 비율 이슈 수월
 - 갤럭시 Z Fold 2를 올릴때 DPM이오전 2021 화면 비율을 지원하지 않던 현상이 수정되었습니다
 - 로그인과 송금 SNS 로그인 양식이 화면 비율에 장황도 없이 잘 맞습니다

메인 / 기획전

- ▶ **[PC/Mobile] 배터리의 사용 전 · 사용 후 관리법/충전법 및 주의**
 - 배터리의 사용 전 관리에 대한 '가이드라인'을 많이 참조하십시오.
 - 충전기 사용 기간(4월 2회) 10A ~ 4월 2회 10A 동안 '충전기'를 많이 활용 하십시오.
- ▶ **[PC/Mobile] 충전기/배터리 관련 주의사항 (핵심)**
 - PC에 상용 온도저장기 안내 문구를 '사용자'로 변경하여 미리 확인 하십시오.
 - PC 마지막 사용 영역의 배정 계층 통신이 수정되었습니다.
- ▶ **[PC/Mobile] 충전기 및 충전 용량**
 - 충전기/배터리 기능에 충포에 대한 배터리 사용 전도 노후 주기적으로 정의되었습니다.
 - 충전기/배터리 - 실시간 전압/전류 측정용 통신을 추가하였습니다.
- ▶ **리튬 이온 배터리의 사용 전 · 사용 후 관리법**
 - 리튬 이온 배터리의 사용 전 · 사용 후 관리에 대한 '가이드라인'을 많이 참조 하십시오.
 - 리튬 이온 배터리의 사용 전 · 사용 후 관리에 대한 '가이드라인'을 많이 참조 하십시오.

3-2. 멀티 AI PR 리뷰 자동화

Problem

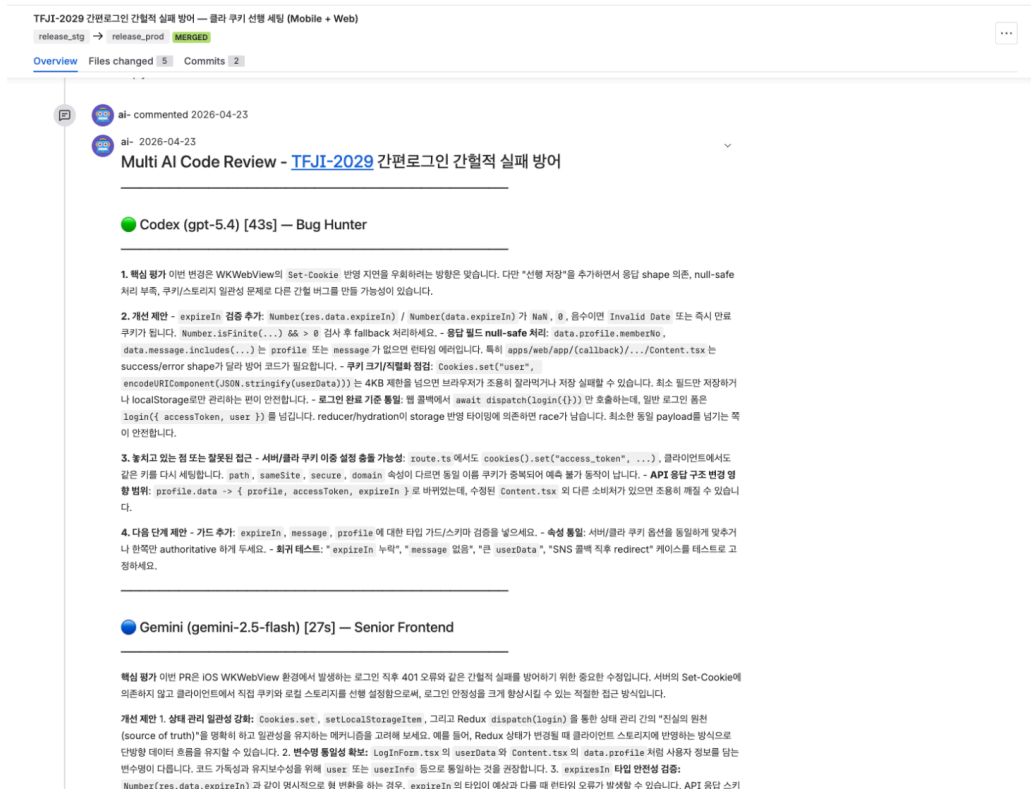
PR 리뷰에서 누락되는 케이스를 줄이고 싶었지만, 사람이 모든 diff를 반복적으로 확인하기에는 시간이 많이 들었습니다. 특히 사소한 타입 오류, 영향 범위 누락, 테스트 관점은 리뷰 전에 미리 점검할 수 있는 구조가 필요했습니다.

Solution

- PR diff를 Codex, Claude, Gemini CLI에 병렬 요청
- 각 도구의 리뷰 결과를 통합해 중복 의견과 유효한 지적을 분리
- 사람 리뷰 전에 사전 점검 도구로 활용

Result

- 10개 PR 에 멀티 AI 리뷰 적용
- 동료 코드 리뷰 전 사전 검토 품질 보완
- 반복적인 diff 확인 비용 감소



멀티 AI PR 리뷰 메달(Bitbucket)

3-3. AI 기반 개발 워크플로우 자동화와 변경 범위 통제

Problem

Claude Code를 티켓 개발에 활용하려면 “코드를 생성하는 것”보다 요구사항 해석, 수정 범위, 검증 누락을 통제하는 구조가 필요했습니다.

단발성 프롬프트로는 티켓마다 결과 품질이 흔들렸고, 사람이 무엇을 기준으로 승인해야 하는지도 불명확했습니다.

Solution

- Node.js 기반 오케스트레이터로 **PLAN → GENERATE → CHECK** 단계를 분리
- PLAN**: Jira 티켓/에픽 분석, 페이지 스펙 확인 또는 자동 생성, 개발 계획 JSON/Markdown 저장, Confluence 게시
- GENERATE**: **production** 기준 worktree 생성, Claude Code 헤드리스 실행, 계획 외 파일 변경 감지, commit/push 자동화
- CHECK**: 사람 리뷰 승인, E2E 필요 여부 확인, **development** rebase/merge/push 흐름 자동화
- plans/{ticketId}.state.json** 으로 단계별 실행 상태를 저장해 중단 후 이어서 실행 가능하도록 설계

- 필수 필드 검증, 보호 브랜치 차단, worktree 격리, 계획 외 파일 변경 확인 등 하네스 규칙 적용

Result

- Jira 티켓 기준 개발 계획 수립부터 코드 생성, 검증, 머지 준비까지 단계화
- 티켓별 개발 계획과 페이지 스펙을 재사용 가능한 문서 자산으로 축적
- 계획 외 파일 변경 감지와 worktree 격리로 AI 코드 생성의 변경 범위 통제
- RCA/스펙/개발 계획서 등 60개 위키 문서 작성 흐름에 활용

Email uwm1004@gmail.com

Github <https://github.com/LeeEugene1>